

Compiler Construction Report

Architecture and Implementation of the Quantum Framework Transpiler
(Qiskit, Cirq, PennyLane to OpenQASM 3.0)

QCanvas Development Team

May 11, 2026

Abstract

This report details the compiler construction principles applied to the transpilation pipeline of the QCanvas platform. The platform implements a source-to-source compiler (transpiler) that translates quantum circuits written in Python dialects of three major quantum frameworks (Qiskit, Cirq, and PennyLane) into standardized OpenQASM 3.0. By framing the transpiler within the classical phases of compiler design—Lexical Analysis, Syntax Parsing, Semantic Analysis, Intermediate Representation (IR), and Target Code Generation—this report provides a comprehensive overview of how Python’s built-in Abstract Syntax Tree (AST) module was leveraged to build a secure, execution-free, multi-framework quantum compiler.

Contents

1	Introduction	3
2	Compiler Architecture Overview	3
3	Phase 1: Lexical Analysis and Syntax Parsing (Frontend)	4
3.1	Lexing and Parsing	4
3.2	Framework-Specific AST Visitors	4
4	Phase 2: Semantic Analysis and Symbol Table	4
4.1	The Symbol Table	4
4.2	Variable Tracking and Type Resolution	4
5	Phase 3: Intermediate Representation (IR)	5
5.1	Structure of the CircuitAST	5
6	Phase 4: Optimization and Code Translation	5
6.1	Gate Mapping and The Gate Library	5
6.2	Expression Evaluation	5
7	Phase 5: Target Code Generation (Backend)	5
7.1	Code Emission Strategy	6

8	Phase 6: Error Handling and Fallback Mechanism	6
8.1	Syntax Errors	6
8.2	Unsupported Instructions	6
9	Conclusion	6

1 Introduction

In traditional software engineering, compilers translate high-level programming languages into machine code. In the quantum computing ecosystem, compilers frequently perform **source-to-source translation (transpilation)** to convert high-level framework-specific code into standard assembly formats.

The QCanvas transpiler translates quantum circuits written in proprietary frameworks—namely **Qiskit (IBM)**, **Cirq (Google)**, and **PennyLane (Xanadu)**—into **OpenQASM 3.0**, an open hardware-agnostic intermediate representation. Unlike standard transpilers that execute user code to generate circuits dynamically, our transpiler acts purely on the static source code to maintain security in cloud-hosted environments.

This report explains how our pipeline mimics a traditional compiler’s phases: from reading the source code to generating the final target assembly.

2 Compiler Architecture Overview

The transpiler follows a multi-pass architecture typical of modern compilers (like LLVM or GCC). The compilation pipeline is structurally separated into a **Frontend** (parsing and semantic analysis), a **Middle-end** (Intermediate Representation), and a **Backend** (code generation).

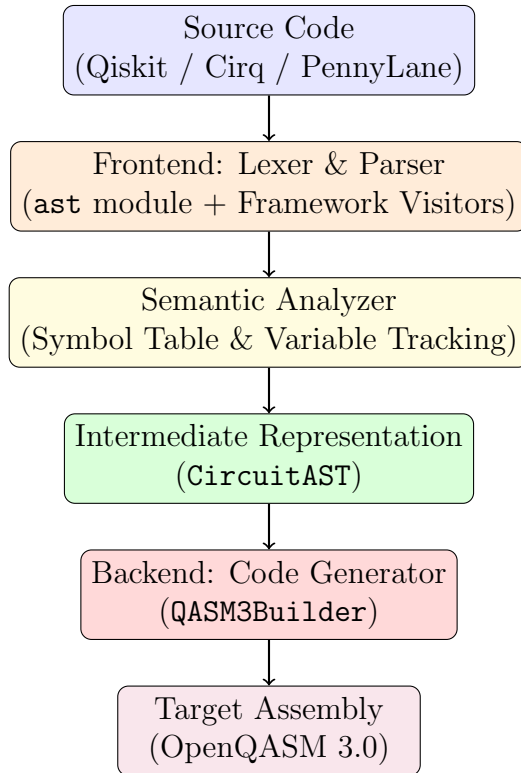


Figure 1: The Multi-Phase Quantum Compiler Pipeline

3 Phase 1: Lexical Analysis and Syntax Parsing (Frontend)

In standard compiler construction (such as with Lex and Yacc), a Lexer tokenizes the input stream and a Parser constructs an Abstract Syntax Tree (AST) based on a defined grammar.

Because the input languages (Qiskit, Cirq, PennyLane) are effectively domain-specific languages (DSLs) embedded within Python, building a custom parser from scratch is redundant and error-prone. Instead, our compiler’s frontend hooks directly into Python’s native `ast` module.

3.1 Lexing and Parsing

The source code is ingested as a string and converted into a generic Python AST using `ast.parse()`. This inherently validates the lexical and syntactic correctness of the Python syntax before any quantum-specific compilation occurs.

3.2 Framework-Specific AST Visitors

Once the Python AST is generated, we employ the **Visitor Pattern** to traverse it. We implemented three separate parsers: `QiskitASTVisitor`, `CirqASTVisitor`, and `PennyLaneASTVisitor`. These visitors extract domain-specific semantics from the generic Python AST nodes.

For example, when traversing Qiskit code, the visitor specifically looks for calls like `qc.h(0)`. In Cirq, it looks for inverted patterns like `cirq.H(q0)`.

4 Phase 2: Semantic Analysis and Symbol Table

Semantic analysis ensures that the parsed syntactic structures have correct meaning and context. In our transpiler, semantic analysis is responsible for variable resolution, resolving dynamic loop bounds, and managing state across the AST traversal.

4.1 The Symbol Table

We maintain a state dictionary (symbol table) mapping variable identifiers to their resolved values. When the visitor encounters an assignment (e.g., `n_qubits = 4`), it stores this mapping.

4.2 Variable Tracking and Type Resolution

Quantum loops heavily utilize variables (e.g., `for i in range(n_qubits): qc.h(i)`). Our semantic analyzer tracks:

- **Loop Variables:** It resolves `i` within the scope of the loop body.
- **Register Dimensions:** It dynamically figures out qubit dimensions if they are assigned to variables instead of integer literals.
- **Classical Bit Mappings:** It analyzes `qc.measure(q, c)` calls to infer how many classical bits are required, assigning them to the correct registers.

5 Phase 3: Intermediate Representation (IR)

To support N source frameworks and M target languages, writing $N \times M$ direct translators is inefficient. Modern compilers solve this by using an Intermediate Representation (IR). We introduced a unified `CircuitAST` as our IR.

5.1 Structure of the CircuitAST

The `CircuitAST` is a framework-agnostic data structure. It isolates the backend code generation from the quirks of the frontend parsers.

```
1 class GateNode:
2     name: str          # e.g., 'h', 'cx', 'rx'
3     qubits: List[int]  # Target qubits
4     parameters: List[Any] # Gate parameters (angles, etc.)
5     modifiers: Dict     # Modifiers like 'ctrl' or 'inv'
```

Listing 1: Simplified representation of the IR Nodes

Every framework parser maps its operations into this unified format. Whether the input is `qc.cx(0, 1)` (Qiskit), `cirq.CNOT(q0, q1)` (Cirq), or `qml.CNOT(wires=[0, 1])` (PennyLane), it becomes a single `GateNode(name='cx', qubits=[0,1])`.

6 Phase 4: Optimization and Code Translation

Before generating the final string assembly, the compiler applies translation passes to optimize and map intermediate code to the exact targets required by OpenQASM 3.0.

6.1 Gate Mapping and The Gate Library

Different frameworks use different names for the same unitary operations. The compiler uses a `QASM3GateLibrary` to standardize gate nomenclature. For example, a `Toffoli` gate in PennyLane and a `CCX` gate in Cirq are both lowered to the `ccx` instruction in OpenQASM.

6.2 Expression Evaluation

Mathematical expressions used in gate parameters (e.g., `np.pi / 2`) are evaluated and formatted through a dedicated `QASM3Expression` module. This ensures that Python-specific references to numpy constants are accurately translated into QASM-native constants (`pi / 2`).

7 Phase 5: Target Code Generation (Backend)

The Backend takes the optimized `CircuitAST` and emits OpenQASM 3.0 syntax. This is managed by the `QASM3Builder`.

7.1 Code Emission Strategy

The builder iterates over the `CircuitAST` and sequentially appends target code instructions:

1. **Prologue:** Emits the file header (`OPENQASM 3.0;`) and necessary include directives (`include "stdgates.inc";`).
2. **Declarations:** Emits qubit and classical bit registers (`qubit[N] q; bit[M] c;`).
3. **Operations:** Translates each `GateNode` into an assembly instruction line, appending OpenQASM gate modifiers (such as `ctrl @` or `inv @`) as required.
4. **Control Flow:** For `ControlFlowNode` elements (like For-loops or If-statements), the builder emits valid OpenQASM 3.0 block structures, applying correct indentation for readability.

8 Phase 6: Error Handling and Fallback Mechanism

A robust compiler must handle ill-formed input gracefully.

8.1 Syntax Errors

If the provided code contains standard Python syntax errors, the parser immediately throws a `SyntaxError` during the AST generation phase, halting compilation.

8.2 Unsupported Instructions

If the semantic analyzer encounters a framework-specific function call that cannot be mapped into a `GateNode` (e.g., deeply nested metaprogramming or unsupported custom operations), it triggers an execution-based fallback. This fallback executes the script in an isolated, sandboxed environment to dynamically serialize the in-memory circuit graph, ensuring the compilation process rarely fails.

9 Conclusion

The transpilation pipeline of the QCanvas platform mirrors the rigorous architecture of a standard compiler. By segregating the logic into a framework-specific frontend, a generic Intermediate Representation (`CircuitAST`), and a unified backend code generator, the system achieves massive scalability. The AST-based approach entirely eliminates the security risks of execution-based compilation, providing a highly reliable and extremely fast conversion framework.